

COS 221 PA 5

Taya Govender - u24697274

Cameron Ambraal - u25008227

Nihaan Bharath – u25044720

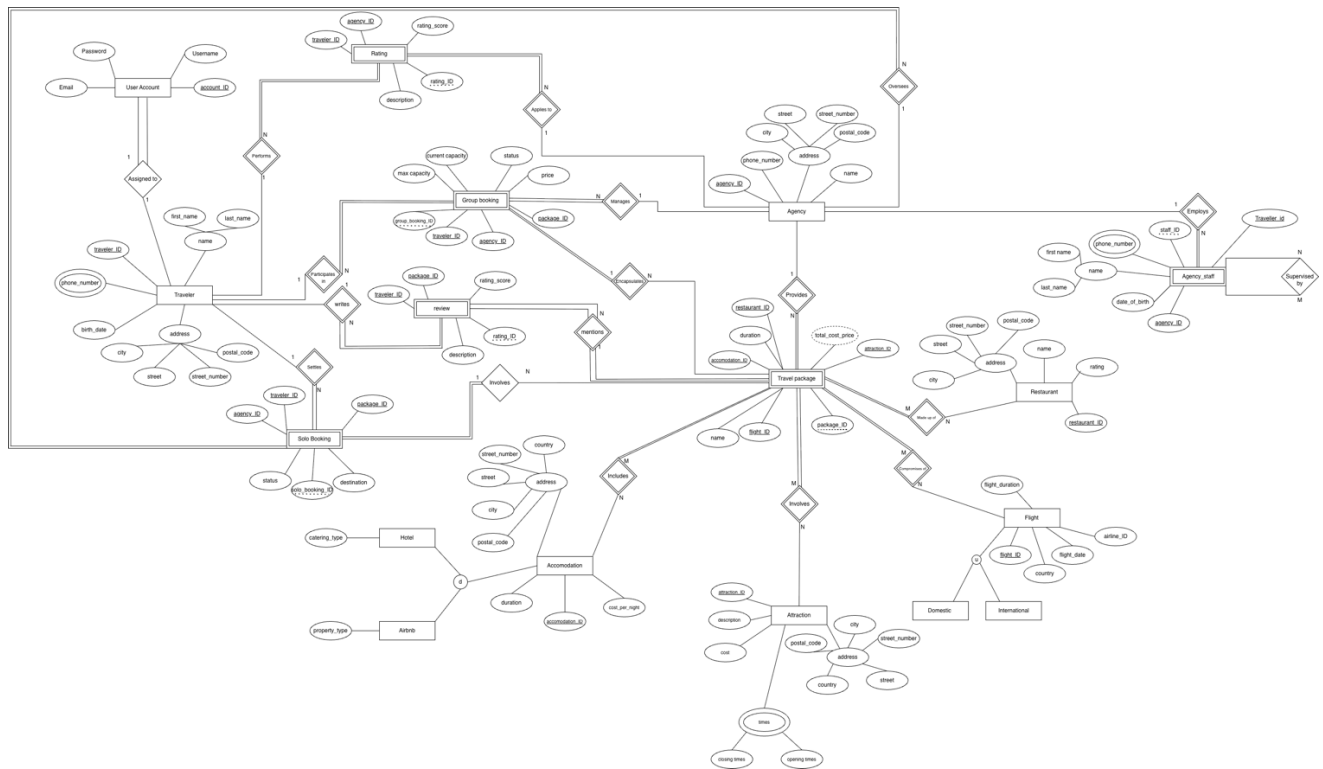
Munashe Chitoro- u24888096

Lizalise Mbonisweni – u23587874

Table of Contents

Task 2: EER Diagram	2
Task 3: EER Diagram to Relational Mapping.....	3
Conversion steps.....	3
Step 1: Mapping of regular (strong) entity types	3
Step 2: Mapping of weak entity types	3
Step 6: Mapping of multivalued attributes	4
Step 7: Mapping of N-ary relationships.....	5
Step 8: Mapping specialisation and generalization.....	5
Task 4: Relation Schema.....	7
Task 5: Web-Based Application	9
Task 6: Data	9
Task 7: Analyse and Optimise.....	10
Task 8: Development	11

Task 2: EER Diagram



Task 3: EER Diagram to Relational Mapping

Conversion steps

Step 1: Mapping of regular (strong) entity types

Each strong entity from the EER diagram was mapped directly to its own relation, with its simple attributes becoming columns and its key attribute becoming the primary key. This produced the following relations for example Traveller (PK: traveler_ID)

Step 2: Mapping of weak entity types

Weak entities were mapped by including the primary key of their owner entity as a foreign key, forming a composite or inherited key. Agency_staff was mapped this way it includes agency_ID as a foreign key referencing Agency, since staff members are dependent on the agency they belong to.

Step 3: Mapping of binary 1:1 relationships

For any 1:1 relationships, the primary key of one side was placed as a foreign key on the other side, preferably on the relation whose entity has total participation in the relationship. In this model, the 1:1 relationship between Agency_staff and Traveller was handled this way traveler_ID was placed as a foreign key inside the Agency_staff relation, referencing the Traveller relation, since each staff member is associated with one traveller.

Step 4: Mapping of binary 1:N relationships

For 1:N relationships, the primary key of the "1" side was added as a foreign key on the "N" side. Examples in the model:

- agency_ID appears in Group_booking, Solo_Booking, and Rating (Agency is the "1" side)
- traveler_ID appears in Group_booking, Solo_Booking, and Rating (Traveller is the "1" side)
- agency_ID appears in Agency_staff

Step 5: Mapping of binary M:N relationships

M:N relationships were resolved by creating a new relation containing the primary keys of both participating entities as foreign keys, which together form the primary key of the new relation. In this model, the M:N relationships included Travel Package to accommodation, Travel Package to restaurant, Travel Package to flight and Travel Package to attraction. This was mapped in the model by creating a new relations for each relationship containing the Primary key Travel Package and the participating entity.

These relations include:

Package_Accommodation

Package_ID | Accommodation_ID

Package_Attraction

Package_ID | Attraction_ID

Package_Flight

Package_ID | Flight_ID

Package_Restaurant

Package_ID | Restaurant

Step 6: Mapping of multivalued attributes

Three multivalued attributes were identified and each was mapped to a separate relation.

Agency_staff_phone : stores multiple phone numbers per agency staff member, with Agency_ID as a foreign key referencing the Agency_Staff relation, and phone_number stored as CHAR.

Traveller_phone : stores multiple phone numbers per traveller, with Traveller_ID as a foreign key referencing the Traveller relation, and phone_number stored as CHAR.

Attraction_times : stores multiple opening and closing times per attraction, with attraction_ID as a foreign key referencing the Attraction relation, and both opening_time and closing_time stored as TIME.

In each case, the primary key of the parent relation is carried as a foreign key into the new relation, linking each phone number or time entry back to its respective entity.

Step 7: Mapping of N-ary relationships

Where more than two entities participated in a single relationship, a new relation was created holding the foreign keys of all participating entities. Travel_package is the result of this step it brings together flight_ID, accomodation_ID, restaurant_ID, and attraction_ID, representing a package that bundles all these components together.

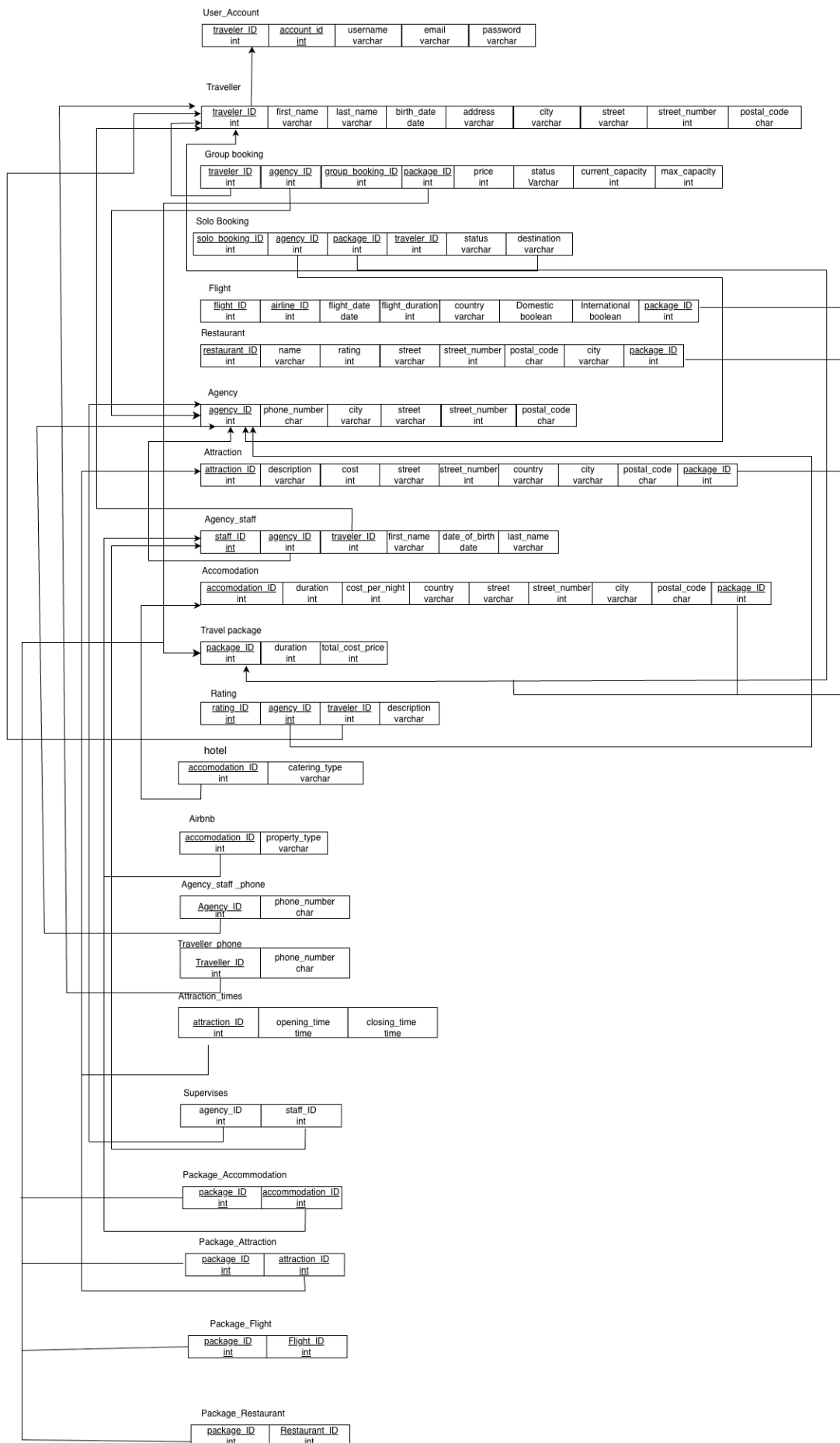
Step 8: Mapping specialisation and generalization

Two specialisation hierarchies from the EER were handled here:

- Accomodation → Hotel / Airbnb (disjoint): Since both subclasses have their own unique attributes (catering_type for Hotel, property_type for Airbnb), each was given its own relation. The accomodation_ID serves as both the primary key and foreign key in the Hotel and Airbnb tables, referencing the Accomodation superclass.
- Flight → Domestic / International: Since neither subclass introduced additional attributes, these were collapsed into the Flight table itself using Domestic and International as columns (type-indicator approach), avoiding unnecessary empty subclass tables.

Step 9: Mapping unions

The u notation on the Flight specialisation in the EER was noted. Since it involved a single superclass with overlapping or union typed subclasses, and no extra attributes existed in the subclasses, this was absorbed into the Flight relation directly as described in Step 8, keeping the schema clean and avoiding redundant tables.



Task 4: Relation Schema

Creation (Entities, Attributes, Constraints, Foreign Keys and Primary Keys)

User_Account(traveler_ID PK FK, account_id PK, username, email, password)

Traveller(traveler_ID PK, first_name, last_name, birth_date, address, city, street, street_number, postal_code)

Group_booking(traveler_ID FK, agency_ID FK, group_booking_ID PK, package_ID FK, price, status, current_capacity, max_capacity)

Solo_Booking(solo_booking_ID PK, agency_ID FK, package_ID FK, traveler_ID FK, status, destination)

Flight(flight_ID PK, airline_ID, flight_date, flight_duration, country, Domestic, International, package_ID FK)

Restaurant(restaurant_ID PK, name, rating, street, street_number, postal_code, city, package_ID FK)

Agency(agency_ID PK, phone_number, city, street, street_number, postal_code)

Attraction(attraction_ID PK, description, cost, street, street_number, country, city, postal_code, package_ID FK)

Agency_staff(staff_ID PK, agency_ID FK, traveler_ID FK, first_name, date_of_birth, last_name)

Accommodation(accommodation_ID PK, duration, cost_per_night, country, street, street_number, city, postal_code, package_ID FK)

Travel_package(package_ID PK, duration, total_cost_price)

Rating(rating_ID PK, agency_ID FK, traveler_ID FK, description)

Hotel(accommodation_ID PK FK, catering_type)

Airbnb(accommodation_ID PK FK, property_type)

Agency_staff_phone(agency_ID FK, phone_number)

Traveller_phone(Traveller_ID FK, phone_number)

Attraction_times(attraction_ID FK, opening_time, closing_time)

Supervises(agency_ID FK, staff_ID FK)

Package_Accommodation(package_ID FK, accommodation_ID FK)

Package_Attraction(package_ID FK, attraction_ID FK)

Package_Flight(package_ID FK, Flight_ID FK)

Package_Restaurant(package_ID FK, Restaurant_ID FK)

Task 5: Web-Based Application

The web-based application was divided into sections with team members working on different components of the application that was later put on Github and integrated together. Tasks were delegated as follows:

- Login functionality as Traveler or Agency: Cameron
- Browsing page for agencies, flights, accommodations, attractions and restaurants: Cameron
- Ratings and reviews page: Taya
- Comparing packages and booking a package: Taya and Cameron
- Creating, deleting, and editing travel packages as Agency: Lizalise
- managing destinations, flights, accommodations, restaurants and attractions associated with a package: Lizalise
- Sort and filter packages: Munashe
- Display a detailed package view: Munashe

Task 6: Data

Maria DB with MySQL workbench was used to implement our data base. We populated all of our tables using a hybrid approach, taking inspiration from flights such as Booking.com, FlightCenter.co.za and HotelTrivago. We decided to populate the method in this way as it is easier for all team members to navigate a database that was created by us making integrating it into our web-application effective. Another reason was that we were able to make the database match our EER exactly as the model was made before the database and tables were created, therefore creating our own tables allowed us to stick to our EER diagram exactly without the need for modifications that would have emerged if we used external existing databases.

Curated Manual Entry: For base core entities: like agency, flight, attraction, and restaurant, we manually inserted high-quality records into our SQL schema. This ensures that when an evaluator uses our presentation frontend, the dropdowns, description text, and target landmark cards display visually perfect, logical records.

Automated Script Seeding: For transactional tables like accomodation, travel_package, and traveller, typing data manually was impossible. We wrote and executed server-side stored procedures (SeedLargeData and FixAccommodationSubclasses) to build **19,906 accommodations** and **55,025 packages**

instantly. Scaling our rows programmatically like this ensures our query optimizer has a substantial production dataset to analyze, making our Task 7 test results meaningful.

Task 7: Analyse and Optimise

1. Scenario and Chosen SQL Query

A frequent search on our Tripistry app happens when a user filters available travel packages by a specific destination city while applying a maximum budget constraint. This operation requires joining our package inventory data with our accommodation location records.

We chose the following query to test and optimize based on the principles in Lectures 28 and 29:

SQL

```
SELECT tp.package_ID, tp.package_name, acc.name AS hotel_name, tp.total_cost_price
FROM travel_package tp
JOIN accomodation acc ON tp.accomodation_ID = acc.accomodation_ID
WHERE acc.city = 'Tokyo' AND tp.total_cost_price <= 120000;
```

2. Baseline Performance Analysis (Pre-Optimisation)

Before applying our secondary indexes, running an EXPLAIN plan showed a major bottleneck in our query execution strategy. Because there was no direct access path for the city or price filters, the query optimizer chose the travel_package table as its starting point, resulting in a **Full Table Scan (type: ALL)** that forced the engine to evaluate all **55,025 rows** sequentially. It then performed a primary key lookup (type: eq_ref) on the accomodation table for every single package row. This unoptimized approach resulted in high disk I/O overhead and a sluggish initial execution runtime.

```
1 EXPLAIN SELECT tp.package_ID, tp.package_name, acc.name AS hotel_name, tp.total_cost_price
2 FROM travel_package tp
3 JOIN accomodation acc ON tp.accomodation_ID = acc.accomodation_ID
4 WHERE acc.city = 'Tokyo' AND tp.total_cost_price <= 120000;$
```

Auto Commit Manual										
	id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	1	SIMPLE	tp	ALL	accomodation_ID	(NULL)	(NULL)	(NULL)	55025	Using where
2	1	SIMPLE	acc	eq_ref	PRIMARY	PRIMARY	4	practical_5_cam.tp.accomodation_ID	1	Using where

3. Proposed Optimisation Implementation

In accordance with Lecture 29's focus on cost estimation and access paths, we created two target secondary B-Tree indexes to give the database engine a non-sequential search route for our WHERE clause filters:

SQL

```
CREATE INDEX idx_accomodation_city ON accomodation(city);  
CREATE INDEX idx_package_price ON travel_package(total_cost_price);
```

4. Post-Optimisation Verification and Analysis

After creating the indexes, re-running our EXPLAIN query proved that the database engine completely overhauled its execution strategy for maximum efficiency.

Instead of sweeping through 55,000+ packages, the optimizer used our new `idx_accomodation_city` index to switch the driving table to `accomodation`. The access path upgraded to a clean index lookup (**type: ref**), instantly isolating just the **4,000 records** that matched 'Tokyo'. From there, it used our relationship keys to look up matching rows in the `travel_package` table, checking a mere **2 rows** per accommodation instead of a massive table scan.

```
1 EXPLAIN SELECT tp.package_ID, tp.package_name, acc.name AS hotel_name, tp.total_cost_price  
2 FROM travel_package tp  
3 JOIN accomodation acc FORCE INDEX (idx_accomodation_city) ON tp.accomodation_ID = acc.accomodation_ID  
4 WHERE acc.city = 'Tokyo' AND tp.total_cost_price <= 120000;
```

Auto Commit Manual

	id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	1	SIMPLE	acc	ref	idx_accomodati...	idx_acc...	403	const	4000	Using inde...
2	1	SIMPLE	tp	ref	accomodation_...	accom...	5	practical_5_cam.acc.accomodation...	2	Using where

Task 8: Development

Link to Github: <https://github.com/TayaGovender/PA5>